

Clase 03. Polimorfismo



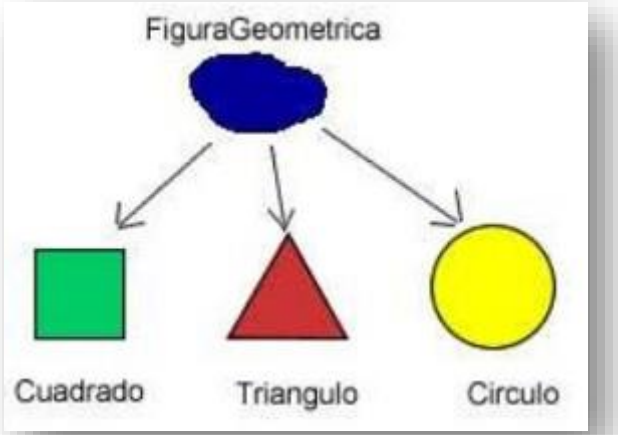
Agenda

- Polimorfismo
- Abstracción
  - Clases abstractas
  - Subclases concretas
  - Métodos y clases finales
- Interfaces



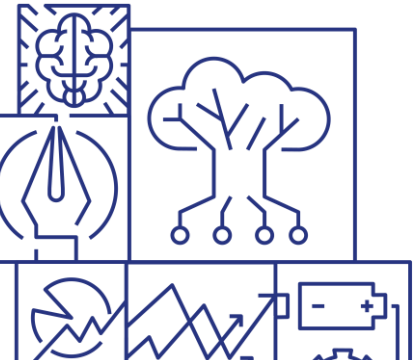
fidélitas
Universidad

Concepto de polimorfismo



- Según (Deitel, 2015), “el polimorfismo permite que diferentes objetos respondan de modo distinto al mismo mensaje”.

- Es una forma de reutilización de software en la que se crea una nueva clase absorbiendo los miembros de una clase existente. El polimorfismo nos permite programar en forma general, en lugar de hacerlo en forma específica.



fidélitas Universidad

Características

1. Nos permite tratar los aspectos más generales del problema.
2. Deja al entorno de ejecución se encargue de los detalles específicos.
3. Podemos indicarle a los objetos que se comporten de una forma determinada.
4. No es necesario conocer el tipo de los objetos, sólo que pertenezcan a la misma jerarquía.





- Consideremos la siguiente jerarquía de clases

Concepto de colección



Según (Joyannes, 2011), “Una colección es una agrupación de objetos relacionados que forman una única entidad.”

Tipos de colecciones

Colecciones de objetos del mismo tipo son llamadas colecciones homogéneas, por ejemplo:

```
MiFecha[] fechas = new MiFecha[2];
fechas[0] = new MiFecha(11,9,2013);
fechas[1] = new MiFecha(15,11,2013);
```



Colecciones de objetos de diferentes clases son llamadas colecciones heterogéneas, por ejemplo:

```
Empleado [] empleado = new Empleado[10];
empleado[0] = new Administrador();
empleado[1] = new Empleado();
empleado[2] = new Programador();
```





fidélitas Universidad

El operador instanceOf

El operador instanceOf permite determinar si un objeto es de un tipo determinado.

```
public class Empleado;
public class Administrador extends Empleado;
public class Programador extends Empleado;
```




```
public void revisaAlgo(Empleado e) {
 if (e instanceof Administrador) {
  //realiza el proceso para administradores
  Administrador a = (Administrador) e;
  System.out.println("Administrador de "+
          a.getDepartment());
 } else if (e instanceof Programador) {
  // realiza el proceso para un programador
 } else {
  // realiza otro proceso
 }
}
```

fidélitas Universidad

Clases abstractas



fidélitas Universidad

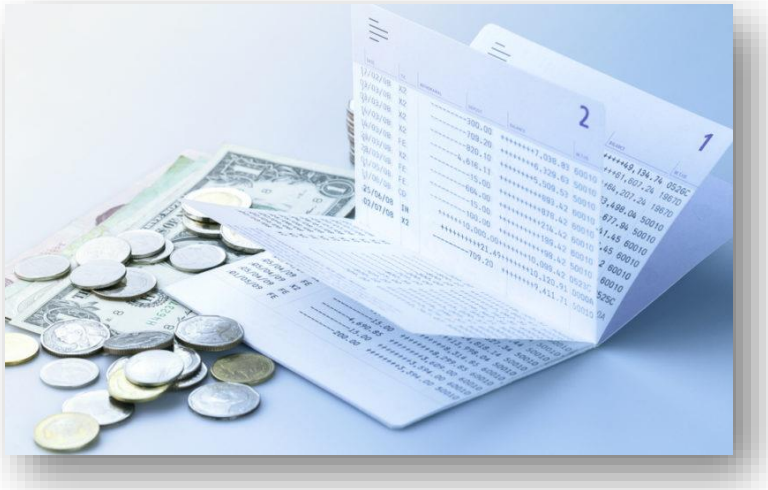


- Conceptos

1. Una clase es más especializada cuanto más alejada está de la clase base.
2. Cuando una clase se diseña para ser genérica, es casi seguro que no necesitaremos crear objetos de ella.
3. Su función es proporcionar los atributos y comportamiento que serán compartidos por todas sus subclases.
4. Una clase puede ser tan genérica que no se puede implementar el comportamiento, sólo se indica y en las subclases se implementa.
5. La clase que se comporta como lo anterior tendrá un método abstracto y la clase por derivación será abstracta.

Ejemplo

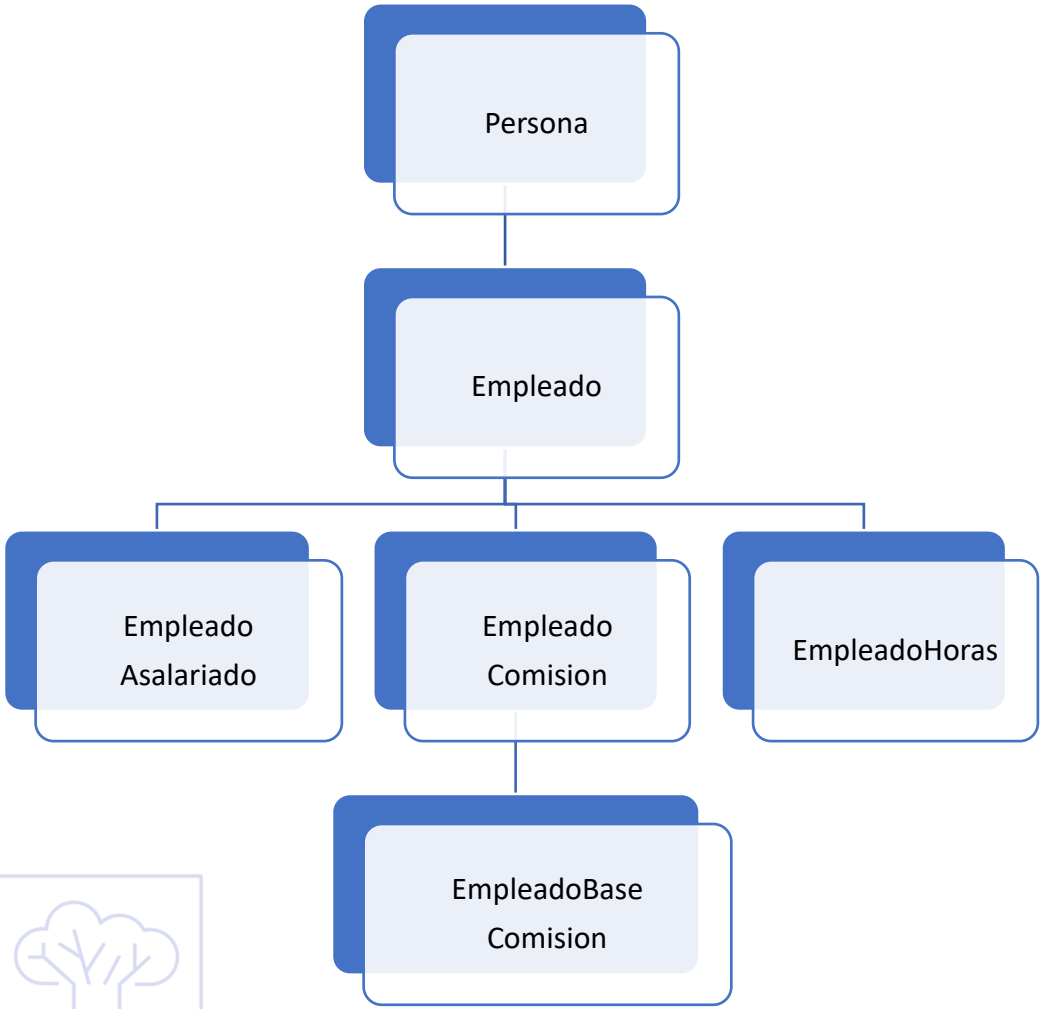
```
public abstract class Cuenta {
  // …
  public abstract void calcularComision();
  // …
}
```



- Una clase abstracta puede contener el mismo tipo de miembros que una clase que no lo sea y además debiera contener métodos abstractos.
- No se permite crear objetos a partir de una clase abstracta.
- No tiene cuerpo, sólo proporciona el comportamiento que debe ser definido en las subclases.

fidElitas Universidad

Consideremos la siguiente jerarquía de clases



Todo empleado percibe un salario, este salario se calcula según el tipo de empleado (objeto), en la clase empleado creamos un método abstracto:

public abstract float calculaSalario();

Este método no tiene cuerpo, esto es porque no se puede saber cómo se calcula el salario, esto se sabrá según el tipo de empleado en las subclases.

Todas las subclases de Empleado deben implementar el método calcularSalario().

fidélitas Universidad

Práctica



- Desarrollemos una aplicación de geometría donde se implemente la clase Forma como clase genérica.
- Forma debe contener los métodos abstractos getArea() y getPerimetro().
- Cree las clases concretas Circulo, Triangulo y Cuadrado donde implemente los métodos abstractos.

fidélitas Universidad

Interfaces



Interface
in Java

fidElitas
Universidad



- Conceptos

1. Una *interface public* es un contrato entre el código cliente y la clase que implementa la interface.
2. Una interface no es una Clase
3. En una interface todos los métodos no tienen una implementación.
4. Muchas clases no relacionadas pueden implementar la misma interfaz.
5. Una clase puede implementar muchas interfaces relacionadas.

Sintaxis de declaración de una interface

<modificador> class *<nombre>* [extends *<superclase>*]
[implements *<interface>* [,*<interface>*]*] {
    cuerpo de la interface
}



fidélitas Universidad

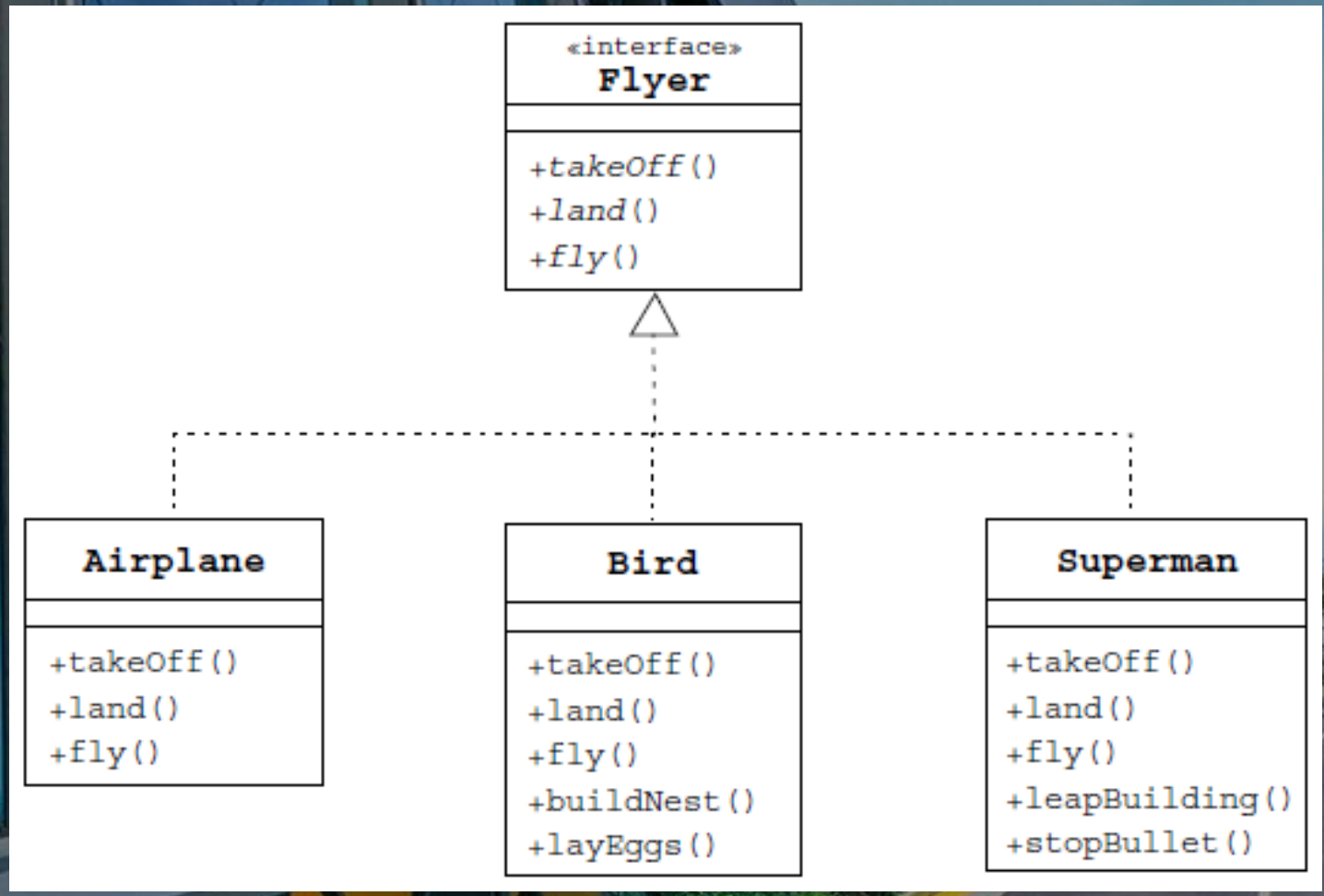
Implementando la interface

```
public class Avion implements Aeronave {
public void despegar() {
//cuerpo del método despegar
}
public void land() {
//cuerpo del método aterrizar
}
public void fly() {
//cuerpo del método volar
}
}
```

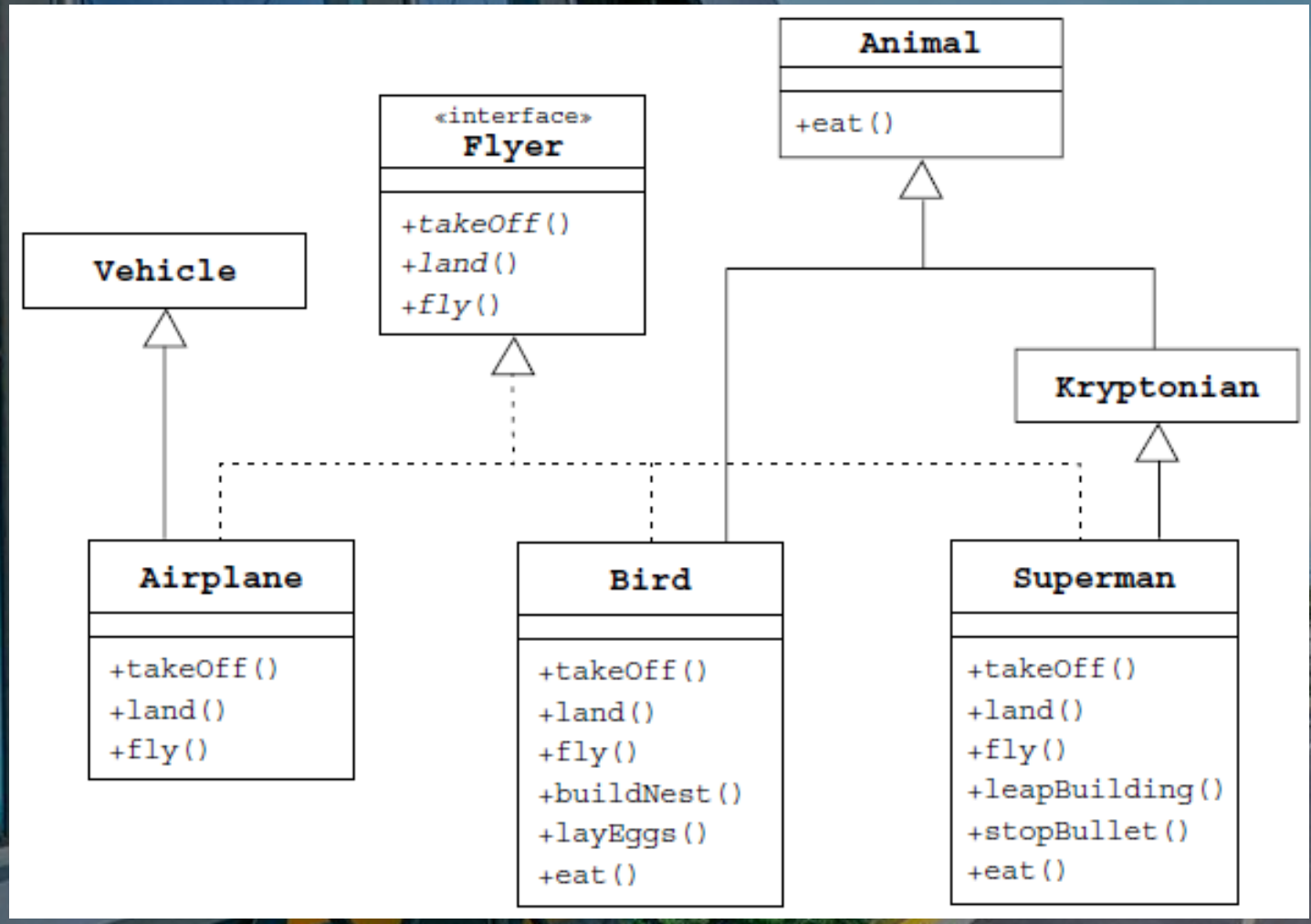


fidélitas Universidad

Jerarquía de clases con interface



Herencia e interfaces



Clases finales



En Java podemos sellar una clase para evitar que otras clases hereden de la misma, colocando la palabra “final” previo a la declaración de la clase.

```
public final class Punto {
   private int x, y;

   public Punto(int x, int y) {
      this.x = x;
      this.y = y;
   }

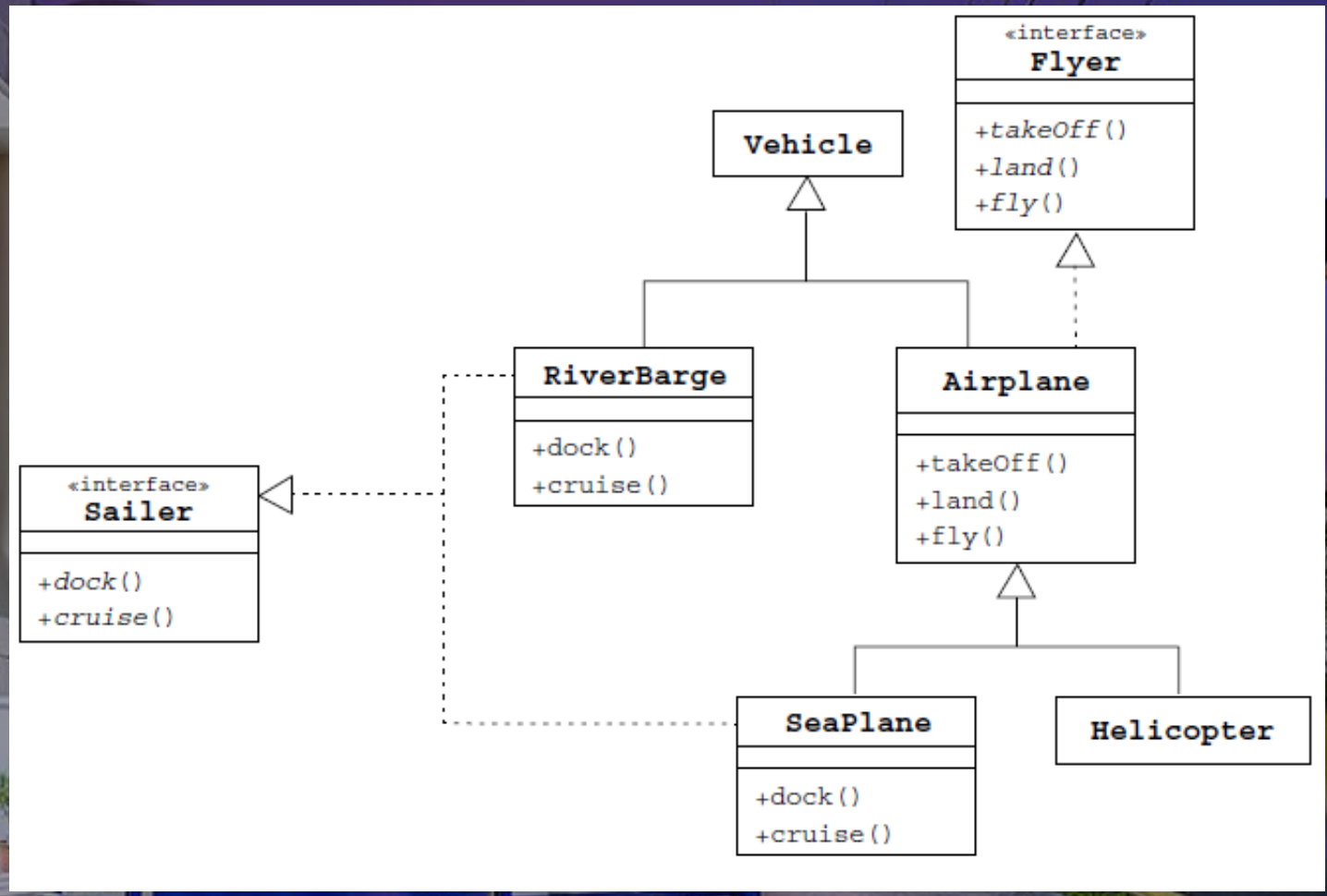
   void imprimirCuadrante() {
      System.out.print("[" + x + "," + y + "] ");
      if (x > 0 && y > 0)
         System.out.println("Se encuentra en el primer cuadrante.");
      else if (x < 0 && y > 0)
         System.out.println("Se encuentra en el segundo cuadrante.");
      else if (x < 0 && y < 0)
         System.out.println("Se encuentra en el tercer cuadrante.");
      else if (x > 0 && y < 0)
         System.out.println("Se encuentra en el cuarto cuadrante.");
      else
         System.out.println("El punto no está en un cuadrante.");
   }
}
```

```
   public static void main(String[] ar) {
      Punto punto1;
      punto1 = new Punto(4, 5);
      punto1.imprimirCuadrante();
      Punto punto2;
      punto2 = new Punto(-4, 5);
      punto2.imprimirCuadrante();

   }
}
```


fidélitas Universidad

Múltiples interfaces



Uso de las interfaces

- Declaración de métodos que una o más clases si requieren su implementación.
- Presentar la interfaz de programación de un objeto sin revelar el código de ésta.
- Capturar similitudes entre clases no relacionadas sin forzar una relación de clase.
- Simular la herencia múltiple declarando una clase que implementa varias interfaces.

interface

fidélitas Universidad



Práctica

Tome la jerarquía de clases de la practica de semana 2 y cree al menos dos clases abstractas con métodos abstractos, utilice al menos una interface.

Si desea puede optar por otra estructura de clases.

fidélitas Universidad

Referencias

Deitel, P. (2015). Cómo Programar en Java. México: Editorial Pearson.

Joyannes, L. (2011). Programación en Java 6. México: Editorial Mc. Graw-Hill.

Hasta la próxima.
Nos vemos.

Tema de la siguiente clase: Excepciones

fidélitas
Universidad